Cache Key Design

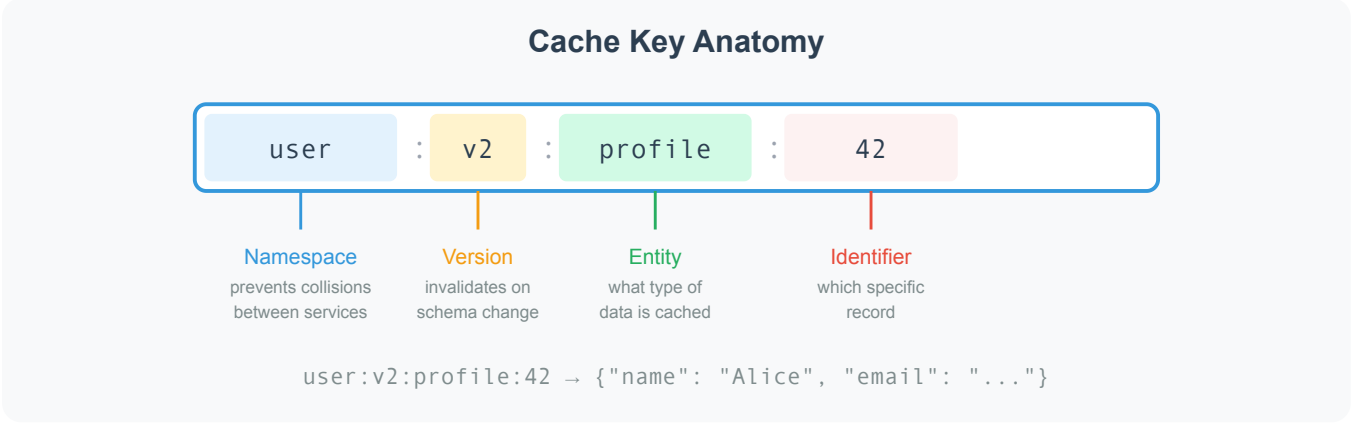
systemdesignschool.io/fundamentals/cache-key-design

The previous articles covered where caches live. Now the question shifts to implementation: when you store data in the cache, what do you use as the key? A poorly designed key leads to collisions (two pieces of data overwriting each other), stale reads after schema changes, and debugging nightmares in production.

Key Structure

A cache key is a string that uniquely identifies a cached value. The simplest approach—using the database primary key directly—fails quickly. If both your user service and your product service cache an item with ID `42`, they collide. One overwrites the other.

A structured key solves this. The standard pattern uses colon-separated segments:



Each segment serves a purpose:

Namespace isolates different services or domains. A user service uses `user:`, a product service uses `product:`. Even within a shared cache cluster, keys never collide across services.

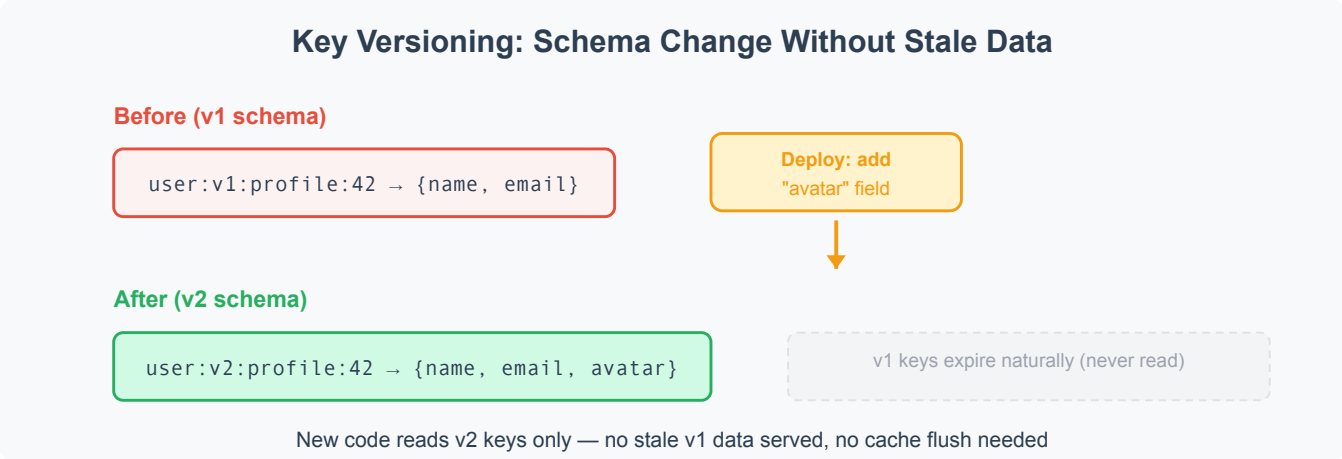
Version enables clean schema migrations (explained in the next section).

Entity identifies what type of data this key holds. A user service might cache profiles, sessions, and preferences separately: `user:v1:profile:42`, `user:v1:session:42`, `user:v1:prefs:42`.

Identifier pinpoints the specific record. Usually the database primary key or a unique identifier like an email or slug.

Versioning

Schema changes break caches silently. Your user profile cached as `{name, email}` gains an `avatar` field in the next deploy. Old cached entries lack this field. The application reads stale data missing the avatar, and users see broken UI until the cache entry expires.



Embedding a version in the key solves this cleanly. When the schema changes, bump the version: `user:v1:profile:42` becomes `user:v2:profile:42`. New code reads only v2 keys. Old v1 entries sit untouched until their TTL expires and the cache evicts them. No cache flush required. No stale data served.

This approach costs slightly more memory during the transition period (both v1 and v2 entries coexist). In practice, the overlap is brief—old entries expire within minutes or hours depending on TTL.

Key Size

Cache keys consume memory. Every key is stored alongside its value. A 200-byte key on a million entries costs at least 200MB for raw key bytes, plus per-entry metadata and allocator overhead (often substantially more). As a rule of thumb, keep keys relatively short—on the order of 100 bytes or less.

Practical guidelines:

- Keep keys short but readable. `u:v2:prof:42` saves bytes but makes debugging harder. `user:v2:profile:42` is clear and still compact.
- Avoid embedding large values in keys. A search query like `search:v1:results:{"query":"long user input...","filters":{"category":"electronics","price_min":10}}` is fragile and wastes space. Hash the query parameters instead: `search:v1:results:sha256_prefix`.
- Use consistent separators. Colons (`:`) are the convention in Redis and most caching libraries. Avoid spaces, newlines, or characters that need escaping.

Common Patterns

Different data types suggest different key structures:

Use Case	Key Pattern	Example
Entity by ID	`service:version:entity:id`	`user:v2:profile:42`
Computed result	`service:version:computation:input_hash`	`search:v1:results:a7f3b2`
Per-user state	`service:version:user:uid:entity`	`cart:v1:user:42:items`
Sorted collection	`service:version:entity:sort_key`	`feed:v1:timeline:42`
Configuration	`config:version:key`	`config:v3:feature_flags`

The pattern stays consistent: namespace the service, version the schema, identify the entity, then the specific record.

What Can Go Wrong

Two failure modes appear most in production:

Key collisions. Two different pieces of data map to the same key. This happens when keys are too generic (`cache:42`) or when hash functions produce duplicates. The fix is explicit namespacing—always include the service and entity type.

Unbounded key spaces. A search cache that stores results for every unique query grows without limit. If users can type arbitrary queries, the key space is infinite. The fix is eviction policies (covered in a later article) combined with careful TTLs—cache search results for minutes, not hours.

From there, read patterns show how the application decides when to read from cache versus database, and how to populate the cache on a miss.